

# **Bottom-Up Dynamic Programming**

Principles, Advantages over Memoization, and Problems

Vardhan Kumar Ray

Chennai Mathematical Institute

March 20, 2026 - LH804

# Outline

- 1 Introduction to Dynamic Programming
- 2 General Bottom-Up DP Template
- 3 Why Bottom-Up DP is Faster
- 4 LCS of Three Strings
- 5 Valid Path in a Grid
- 6 Longest Increasing Subsequence (LIS)
- 7 Minimum Cuts for Palindrome Partitioning
- 8 Summary

# What is Dynamic Programming?

## Core Idea

Break a problem into **overlapping subproblems**, solve each *once*, and store the results to avoid redundant computation.

## Two necessary properties:

- ① **Optimal Substructure** - an optimal solution is built from optimal solutions of subproblems.
- ② **Overlapping Subproblems** - the same subproblems recur many times.

# Three Approaches: Recursion, Memoization, Bottom-Up

## Plain Recursion

- Re-solves same subproblems
- Exponential time in general
- Simple to write

## Memoization (Top-Down)

- Recursion
- Call-stack overhead
- Risk of stack overflow

## Bottom-Up DP

- Iterative; no recursion
- Fills table in dependency order
- Enables space optimisation

# General Bottom-Up DP Template

- 1 **Define the DP state**  $dp[i], dp[i][j], \dots$
- 2 **Identify base cases** smallest subproblems whose answers are known.
- 3 **Write the transition** express  $dp[\dots]$  in terms of smaller indices.
- 4 **Determine computation order** ensure each  $dp[\dots]$  is computed *after* all values it depends on.
- 5 **Extract the final answer** read off  $dp[\dots]$  at the target state.

## Mini-Example: Fibonacci

$$dp[0] = 0, \quad dp[1] = 1 \quad dp[i] = dp[i-1] + dp[i-2], \quad i \geq 2$$

Fill left-to-right; answer is  $dp[n]$ . Time:  $O(n)$ , Space:  $O(1)$  after rolling-array optimisation.

# Why Bottom-Up DP is Often Faster than Memoization

## Advantages

- **No function-call overhead**  
each state costs  $O(1)$  extra work (loop counter only).
- **No stack overflow**  
recursion depth can be  $O(n)$  or  $O(n^2)$ ; iterative loops never overflow.
- **Space optimisation**  
rolling arrays or pointer swaps discard rows not needed for future transitions.

## When Memoisation Wins

- Only a *sparse* subset of states is reachable (e.g. tree DP with pruning).
- Cleaner code matters more than constant-factor speed.

## Rule of Thumb

If **most or all states** are needed, bottom-up is faster in practice. If only a **small fraction** of states is visited, memoisation avoids unnecessary computation.

# LCS of Three Strings: Problem Statement

## Problem

Given three strings  $A$ ,  $B$ ,  $C$  of lengths  $m$ ,  $n$ ,  $p$  respectively over an alphabet  $\Sigma$ , find the length of their **Longest Common Subsequence (LCS)**.

**Definitions.** A *subsequence* of a string  $S$  is obtained by deleting zero or more characters from  $S$  without changing the relative order of the remaining characters. A string  $T$  is a *common subsequence* of  $A$ ,  $B$ ,  $C$  if  $T$  is a subsequence of each of  $A$ ,  $B$ , and  $C$ .

**Example.**

$$A = \text{AGCT}, \quad B = \text{GACT}, \quad C = \text{ACGT} \implies \text{LCS}(A, B, C) = \text{ACT}, \quad |\text{LCS}| = 3$$

**Output.** The length of the LCS (and optionally the subsequence itself via backtracking).

## Why DP?

Brute-force over all subsequences of  $A$  costs  $O(2^m)$  - exponential. The three-string LCS has *overlapping subproblems* in three dimensions.

# LCS of Three Strings: DP Formulation

## State

$dp[i][j][k]$  = length of LCS of  $A[1..i]$ ,  $B[1..j]$ ,  $C[1..k]$ .

## Base Cases

$dp[0][j][k] = dp[i][0][k] = dp[i][j][0] = 0 \quad \forall i, j, k$

## Transition

$$dp[i][j][k] = \begin{cases} dp[i-1][j-1][k-1] + 1 & \text{if } A[i] = B[j] = C[k] \\ \max(dp[i-1][j][k], dp[i][j-1][k], dp[i][j][k-1]) & \text{otherwise} \end{cases}$$

**Order:** ★

**Answer:**  $dp[m][n][p]$ .

# LCS of Three Strings: Complexity & Backtracking

## Complexity

- **Time:**  $O(m \cdot n \cdot p)$
- **Space:**  $O(m \cdot n \cdot p)$  for full table
- **Space:** ★ with rolling-layer optimisation (keep only two  $j \times k$  layers)

## Bottom-Up Advantage

Memoisation incurs a recursive call per state; with  $m \cdot n \cdot p$  up to  $10^6$  states the call-stack depth can be  $\sim 100^3$ , bottom-up avoids this entirely.

## Backtracking

Starting from  $(m, n, p)$ , trace backwards:

- If  $A[i] = B[j] = C[k]$ : append  $A[i]$ , go to  $(i - 1, j - 1, k - 1)$ .
- Else move to the neighbour with maximum value among  $(i - 1, j, k)$ ,  $(i, j - 1, k)$ ,  $(i, j, k - 1)$ .
- Stop when any index reaches 0.

Reverse the appended characters to get the LCS string.

**Backtracking time:** ★.

# LCS of Three Strings: Backtracking Pseudocode

**Function** `LCS_Backtrack( $A, B, C, dp, m, n, p$ ):`

$LCS \leftarrow$  empty list

$i \leftarrow m, j \leftarrow n, k \leftarrow p$

**while**  $i > 0$  *and*  $j > 0$  *and*  $k > 0$  **do**

**if**  $A[i] = B[j] = C[k]$  **then**

            append  $A[i]$  to  $LCS$

$i \leftarrow i - 1$

$j \leftarrow j - 1$

$k \leftarrow k - 1$

**else**

**end**

**end**

**while** *continuation* **do**

**if**  $dp[i - 1][j][k] \geq dp[i][j - 1][k]$

*and*  $dp[i - 1][j][k] \geq dp[i][j][k - 1]$  **then**

$i \leftarrow i - 1$

**else**

**if**  $dp[i][j - 1][k] \geq dp[i][j][k - 1]$  **then**

$j \leftarrow j - 1$

**else**

$k \leftarrow k - 1$

**end**

**end**

**end**

    reverse( $LCS$ )

**return**  $LCS$

# Valid Path in a Grid: Problem Statement

## Problem

Given an  $m \times n$  grid where each cell  $(i, j)$  is either *free* (value 0) or *blocked* (value 1), determine whether a valid path exists from the top-left cell  $(1, 1)$  to the bottom-right cell  $(m, n)$ . **Movement:** at each step you may move *right* or *down* only.

**Extended variant (count / minimum-cost path).**

- 1 *Count* the number of distinct valid paths.
- 2 If each free cell carries a non-negative cost  $w(i, j)$ , find the path of *minimum total cost*.

**Example.**

$$\text{Grid} = \begin{pmatrix} 0 & 0 & 1 \\ 0 & 0 & 0 \\ 1 & 0 & 0 \end{pmatrix} \implies \text{valid paths exist: } (1, 1) \rightarrow (2, 1) \rightarrow (2, 2) \rightarrow (2, 3) \rightarrow (3, 3)$$

**Why DP?** Paths share prefixes; the number of paths reaching  $(i, j)$  depends only on the number reaching  $(i - 1, j)$  and  $(i, j - 1)$  (overlapping substructure).

# Valid Path in a Grid: DP Formulation

State (reachability / count)

$dp[i][j]$  = number of valid paths from  $(1, 1)$  to  $(i, j)$ .

Base Cases

$dp[1][1] = 1$  (if cell is free)

First row/column:  $dp[1][j] = dp[1][j - 1]$  (and similarly for  $dp[i][1]$ ).

Transition

$$dp[i][j] = \begin{cases} 0 & \text{if grid}[i][j] = 1 \text{ (blocked)} \\ dp[i - 1][j] + dp[i][j - 1] & \text{otherwise} \end{cases} \quad i, j \geq 2$$

**Minimum-cost variant:** replace addition with  $\min(dp[i - 1][j], dp[i][j - 1]) + w(i, j)$ .

**Order:**  $\star$ ; **Answer:**  $dp[m][n]$  (0 means no valid path).

# Valid Path in a Grid: Complexity & Path Reconstruction

## Complexity

- **Time:**  $O(m \cdot n)$
- **Space:**  $O(m \cdot n)$  full table
- **Space:**  $O(n)$  with single-row rolling array (since  $dp[i][j]$  depends only on row  $i$  and  $i - 1$ )

## Bottom-Up Advantage

Every cell is processed exactly once in row-major order, perfect sequential memory access.

## Path Reconstruction

After filling the table, trace back from  $(m, n)$ :

- At  $(i, j)$ : move *up* to  $(i - 1, j)$  if  $dp[i - 1][j] > 0$ , else move *left* to  $(i, j - 1)$ .
- (Minimum-cost variant: move to the neighbour that gave the minimum.)
- Continue until  $(1, 1)$  is reached.
- Reverse the recorded path.

**Reconstruction time:**  $O(m + n)$ .

# Valid Path in a Grid: Backtracking Pseudocode

**Function** ReconstructPath( $dp, m, n$ ):

```
    if  $dp[m][n] = 0$  then  
        | return NO PATH  
    end  
     $path \leftarrow$  empty list  
     $i \leftarrow m, j \leftarrow n$ 
```

```
    while  $(i, j) \neq (1, 1)$  do  
        append  $(i, j)$  to  $path$   
        if  $i > 1$  and  $dp[i-1][j] > 0$  then  
            |  $i \leftarrow i - 1$   
        end  
        else if  $j > 1$  and  $dp[i][j-1] > 0$  then  
            |  $j \leftarrow j - 1$   
        end  
        else  
            | return NO PATH  
        end  
    end  
    append  $(1, 1)$  to  $path$   
    reverse( $path$ )  
    return  $path$ 
```

# Longest Increasing Subsequence: Problem Statement

## Problem

Given an array  $A = [a_1, a_2, \dots, a_n]$  of  $n$  integers, find the length of a **longest strictly increasing subsequence** (LIS) of  $A$ .

**Definition.** A subsequence  $a_{i_1}, a_{i_2}, \dots, a_{i_k}$  with  $i_1 < i_2 < \dots < i_k$  is *strictly increasing* if  $a_{i_1} < a_{i_2} < \dots < a_{i_k}$ .

**Example.**

$$A = [3, 1, 4, 1, 5, 9, 2, 6] \implies \text{LIS} = [1, 4, 5, 9] \text{ or } [1, 4, 5, 6], \quad |\text{LIS}| = 4$$

**Output.** The length of a LIS (and optionally the subsequence itself).

## Why DP?

The LIS ending at index  $i$  is built by extending the LIS ending at some earlier index  $j < i$  with  $a_j < a_i$ : (optimal substructure).

# LIS: $O(n^2)$ Bottom-Up DP

## State

$dp[i]$  = length of the LIS ending at index  $i$ .

## Base Case

$dp[i] = 1$  for all  $i$  (every single element is an LIS of length 1).

## Transition

$$dp[i] = 1 + \max\{ dp[j] \mid j < i, a_j < a_i \}, \quad i = 1, \dots, n$$

(take  $\max = 0$  if no such  $j$  exists).

**Order:**  $i = 1, 2, \dots, n$  (left-to-right).    **Answer:**  $\max_{1 \leq i \leq n} dp[i]$ .

## Complexity

**Time:**  $O(n^2)$ ;    **Space:**  $O(n)$

# LIS: $O(n \log n)$ via Binary Search

## Key Insight

Maintain an auxiliary array `tails` where `tails[k]` stores the *smallest possible tail element* of any increasing subsequence of length  $k + 1$  seen so far.

## Algorithm

For each  $a_i$ :

- 1 Binary-search `tails` for the *leftmost* position  $\ell$  with `tails[ $\ell$ ]`  $\geq a_i$ .
- 2 Replace `tails[ $\ell$ ]`  $\leftarrow a_i$ .
- 3 If no such  $\ell$  exists, append  $a_i$ : this *extends* the LIS length.

**Answer:** length of `tails` at the end.

## Complexity

**Time:**  $O(n \log n)$

**Space:**  $O(n)$

## DP Insight $\Rightarrow$ Optimisation

The  $O(n^2)$  DP's bottleneck is “find the best  $j < i$ ”.

Maintaining `tails` as a sorted structure turns this into a binary search.

# LIS: $O(n \log n)$ via Binary Search

```
Function LIS( $A[1..n]$ ):  
  tails  $\leftarrow$  empty list  
  for  $i \leftarrow 1$  to  $n$  do  
     $x \leftarrow A[i]$   
     $\ell \leftarrow$  leftmost index in tails with  $\text{tails}[\ell] \geq x$   
    if  $\ell$  exists then  
      | tails[ $\ell$ ]  $\leftarrow x$   
    else  
      | append  $x$  to tails  
    end  
  end  
  return length(tails)
```

## How to find $\ell$

Since tails is always sorted:

- use binary search,
- find the first value  $\geq x$ ,
- replace it with  $x$ .

## Complexity

**Time:**  $O(n \log n)$

**Space:**  $O(n)$

# LIS: Reconstruction

## Approach (using the $O(n^2)$ table)

- 1 Let  $L = \max_i dp[i]$  be the LIS length.
- 2 Find the rightmost index  $i^*$  with  $dp[i^*] = L$ .
- 3 Walk backwards: find the rightmost  $j < i^*$  with  $dp[j] = L - 1$  and  $a_j < a_{i^*}$ ; record  $a_j$ ; update  $i^* \leftarrow j$ ,  $L \leftarrow L - 1$ .
- 4 Repeat until  $L = 0$ . Reverse the recorded sequence.

**Reconstruction time:**  $O(n)$  per step  $\Rightarrow O(n^2)$  total (or  $O(n)$  with a parent pointer array).

## Parent Pointer

Maintain  $\text{parent}[i] = \text{the index } j \text{ that achieved } dp[i] = dp[j] + 1$ . Then trace  $i^* \rightarrow \text{parent}[i^*] \rightarrow \dots$  in  $O(L)$  time.

# Palindrome Partitioning: Problem Statement

## Problem

Given a string  $S$  of length  $n$ , partition  $S$  into substrings such that **every substring is a palindrome**, and the number of cuts is **minimised**.

**Definitions.** A *partition* of  $S$  into parts  $S[i_0..i_1 - 1], S[i_1..i_2 - 1], \dots$  requires  $i_0 = 0$  and the last part ends at  $n - 1$ . The *number of cuts* equals the number of parts minus one.

**Example.**

$S = \text{"aab"} \implies$  partition  $[\text{"aa"}, \text{"b"}]$  has 1 cut (minimum).

$S = \text{"aaaa"} \implies$  partition  $[\text{"aaaa"}]$  has 0 cuts (minimum).

## Why DP?

The minimum cuts for  $S[0..i]$  is determined by the minimum cuts for  $S[0..j - 1]$  over all  $j \leq i$  such that  $S[j..i]$  is a palindrome (optimal substructure with overlapping subproblems).

# Palindrome Partitioning: Precomputation

## Step 1: Precompute Palindrome Table

Let  $\text{isPalin}[i][j] = 1$  iff  $S[i..j]$  is a palindrome.

## Base Cases

$$\text{isPalin}[i][i] = 1 \quad \forall i \quad \text{isPalin}[i][i+1] = 1[S[i] = S[i+1]]$$

## Transition (by increasing length $\ell = j - i + 1 \geq 3$ )

$$\text{isPalin}[i][j] = 1[S[i] = S[j]] \wedge \text{isPalin}[i+1][j-1]$$

**Order:** iterate over *length*  $\ell = 1, 2, \dots, n$ ; for each  $\ell$ , iterate over start index  $i = 0, \dots, n - \ell$ .

## Complexity of Precomputation

**Time:**  $O(n^2)$       **Space:**  $O(n^2)$

# Palindrome Partitioning: Cuts DP

## State

$dp[i]$  = minimum number of cuts in  $S[0..i]$ .

## Base Case

$$dp[0] = 0$$

## Transition

$$dp[i] = \min_{0 \leq j \leq i, \text{isPalin}[j][i]} (dp[j - 1] + 1) \quad (\text{convention: } dp[-1] = -1)$$

**Order:**  $i = 0, 1, \dots, n - 1$ . **Answer:**  $dp[n - 1]$ .

## Total Complexity

**Time:**  $O(n^2)$ ; **Space:**  $O(n^2)$  for isPalin;  
 $O(n)$  for dp

## vs. Naive Recursion

Naive recursion without memoisation tries all  $O(2^n)$  partitions. Bottom-up DP reduces this to polynomial time with zero stack overhead.

# Key Patterns Learned

## Core Problems Covered

| Problem               | State Dim. | Complexity    | Key Idea                         |
|-----------------------|------------|---------------|----------------------------------|
| LCS of 3 Strings      | 3-D        | $O(mnp)$      | Match all 3 chars simultaneously |
| Valid Path in Grid    | 2-D        | $O(mn)$       | Count/cost of reachable paths    |
| LIS ( $O(n^2)$ )      | 1-D        | $O(n^2)$      | Extend best ending at $j < i$    |
| LIS ( $O(n \log n)$ ) | 1-D        | $O(n \log n)$ | Binary search on tails array     |
| Palindrome Partition  | 1-D + 2-D  | $O(n^2)$      | Precompute palindromes first     |

# Final Takeaway

- 1 **Think in states and transitions**, the mathematical structure is identical to memoisation; only the *execution order* differs.
- 2 **Always exploit the filling order** to enable rolling-array space optimisations.
- 3 **The  $O(n^2) \rightarrow O(n \log n)$  LIS trick** shows that DP insight drives algorithmic optimisation beyond mere memoisation.
- 4 **Precompute auxiliary tables** (e.g. `isPalin`) when the main transition relies on an expensive sub-query.

*“First make it correct (states + transitions), then make it fast (order + space).”*